⚡ WebClient & Reactive Streams

Complete Guide — Spring Boot Microservices

> Covers every WebClient pattern from first principles to production-ready parallel calls, error handling, and Spring Boot integration. Written for developers moving from RestTemplate to reactive programming.

📋 Table of Contents

1. What is WebClient and Why Use It?
2. Mono and Flux — The Building Blocks
3. Setup and Configuration
4. GET Requests
5. POST, PUT, PATCH, DELETE
6. Core Operators
7. Error Handling
8. Parallel Calls
9. WebClient in Spring Boot
10. Testing Reactive Code
11. Best Practices Cheat Sheet

---

1. What is WebClient and Why Use It?

RestTemplate vs WebClient — the core difference

`RestTemplate` (the old way) **blocks** the calling thread while waiting for an HTTP response. The

thread sits idle, doing nothing, until the remote service replies.

`WebClient` (the new way) is **non-blocking** — the thread is freed the instant the request is sent and picked up again only when the response actually arrives. This allows handling far more concurrent requests with the same number of threads.

```
RestTemplate — blocking:



  Thread A → sends request → BLOCKED (waiting 300ms) → gets response → continues

               ↑ thread is useless for 300ms



WebClient — non-blocking:



  Thread A → sends request → immediately freed → handles other requests

                                          ↓

                              300ms later: response arrives → Thread A (or any thread)

                              picks it up and continues
```

Why this matters for microservices

In a microservices system, every service calls other services. If each call blocks a thread for 300ms and you have 200 threads, your service can only handle 200 concurrent requests. With WebClient and Reactor's non-blocking I/O, the same 8 event-loop threads can handle thousands of concurrent requests.

	RestTemplate	WebClient
Thread model	Blocking (one thread per request)	Non-blocking (event loop)
Concurrency	~200 (thread pool size)	Thousands
Spring version	Spring 3+	Spring 5+
Status	Maintenance mode	Actively developed
Returns	The value directly	`Mono<T>` or `Flux<T>`
Reactive support	No	Yes

The key mental model

```
// RestTemplate — you GET the value immediately (thread blocked)

UserResponse user = restTemplate.getForObject(

    "http://user-service/api/users/1", UserResponse.class);

// user is available right here, but your thread was blocked for 300ms



// WebClient — you get a RECIPE for getting the value

Mono<UserResponse> userMono = webClient

    .get()

    .uri("http://user-service/api/users/1")

    .retrieve()

    .bodyToMono(UserResponse.class);

// Returns instantly. Nothing has happened yet.
```

```
// The HTTP call fires when something subscribes to userMono.
```

A `Mono` or `Flux` is a **pipeline definition**, not a result. Nothing executes until something subscribes to it. In Spring WebFlux, the framework subscribes for you when you return a `Mono` from a controller method.

---

2. Mono and Flux — The Building Blocks

Mono\<T> — exactly 0 or 1 item

`Mono` represents an asynchronous operation that will eventually produce at most one value (or nothing, or an error).

```
// Empty Mono — like Optional.empty()

Mono<String> empty = Mono.empty();




// Mono with a single value

Mono<String> hello = Mono.just("Hello World");




// Subscribe to receive the value

hello.subscribe(

    value -> System.out.println("Got: "   + value),   // onNext

    error -> System.err.println("Error: " + error),   // onError
```

```
    ()    -> System.out.println("Done!")               // onComplete

);

// Output:

// Got: Hello World

// Done!




// Common Mono uses with WebClient:

Mono<UserResponse>  user    =
webClient.get()...bodyToMono(UserResponse.class);

Mono<Void>          deleted = webClient.delete()...bodyToMono(Void.class);

Mono<UserResponse>  created =
webClient.post()...bodyToMono(UserResponse.class);
```

Flux\<T> — 0 to N items (a stream)

`Flux` represents an asynchronous sequence that can emit zero, one, or many values over time.

```
// Create from known values

Flux<String> fruits = Flux.just("apple", "banana", "cherry");




// Create from a List

var names = List.of("Alice", "Bob", "Carol");
```

```
Flux<String> nameFlux = Flux.fromIterable(names);



// Subscribe — onNext is called once per item

fruits.subscribe(

    item  -> System.out.println("Item: " + item),

    error -> System.err.println("Error!"),

    ()    -> System.out.println("Stream complete!")

);

// Output:

// Item: apple

// Item: banana

// Item: cherry

// Stream complete!



// Common Flux uses with WebClient:

Flux<UserResponse>    users  =
webClient.get()...bodyToFlux(UserResponse.class);

Flux<ServerSentEvent> events =
webClient.get()...bodyToFlux(ServerSentEvent.class);
```

When to use Mono vs Flux

Use Mono when...	Use Flux when...
`GET /users/42` → one user	`GET /users` → list of users
`POST /users` → created user	Server-sent events stream
`DELETE /users/42` → Void	WebSocket messages
Checking existence → Boolean	Streaming large datasets
Single aggregation (count, sum)	File upload/download chunks

The reactive stream lifecycle

```
Publisher (Mono/Flux)    Subscriber (your code / Spring)

          |                          |

          |◀── subscribe() ──────────┘

          |

          |── onSubscribe(subscription) ──▶|

          |                                |

          |── onNext(item1) ──────────────▶|  ← item arrives

          |── onNext(item2) ──────────────▶|  ← next item

          |         ...                    |

          |── onComplete() ───────────────▶|  ← all done

          |    (or onError(e) on failure)  |
```

---

3. Setup and Configuration

Maven dependency

```
<!-- Option 1: Full reactive stack (WebFlux) -->

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-webflux</artifactId>

</dependency>



<!-- Option 2: Keep Spring MVC but add WebClient -->

<!-- Add both — Spring MVC for controllers, WebClient for HTTP calls -->

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-webflux</artifactId>

</dependency>
```

WebClient bean configuration

```
@Configuration

public class WebClientConfig {



    @Bean

    public WebClient webClient() {

        return WebClient.builder()

                // Base URL — all calls in this client are relative to this

                .baseUrl("http://user-service")



                // Default headers sent on every request

                .defaultHeader(HttpHeaders.CONTENT_TYPE,

                               MediaType.APPLICATION_JSON_VALUE)

                .defaultHeader(HttpHeaders.ACCEPT,

                               MediaType.APPLICATION_JSON_VALUE)



                // Increase memory limit (default is 256 KB — too small for
large lists)

                .codecs(config -> config
```

```
                .defaultCodecs()
                .maxInMemorySize(1 * 1024 * 1024))  // 1 MB


            // Connection and read timeouts
            .clientConnector(new ReactorClientHttpConnector(
                HttpClient.create()
                    .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 5000)
                    .responseTimeout(Duration.ofSeconds(10))
            ))
            .build();
}


// Multiple clients — one per downstream service
@Bean("budgetWebClient")
public WebClient budgetWebClient() {
    return WebClient.builder()
            .baseUrl("http://budget-service")
            .defaultHeader(HttpHeaders.CONTENT_TYPE,
                           MediaType.APPLICATION_JSON_VALUE)
```

```
                .codecs(config -> config.defaultCodecs()

                    .maxInMemorySize(512 * 1024))  // 512 KB

                .build();

    }

}
```

Inject and use in a service

```
@Service

@RequiredArgsConstructor

public class UserServiceClient {



    private final WebClient webClient;  // injected from @Bean above



    public Mono<UserResponse> getUser(Long userId) {

        return webClient

                .get()

                .uri("/api/v1/users/{id}", userId)

                .retrieve()

                .bodyToMono(UserResponse.class);
```

```
    }

}
```

application.yml settings

```
spring:

  webflux:

    base-path: /api




# Logging reactive requests (useful for debugging)

logging:

  level:

    reactor.netty.http.client: DEBUG

    org.springframework.web.reactive: DEBUG
```

---

4. GET Requests

Basic GET — single object

```
public Mono<UserResponse> getUserById(Long id) {

    return webClient
```

```
            .get()

            .uri("/api/v1/users/{id}", id)

            .retrieve()

            .bodyToMono(UserResponse.class);

}
```

GET — list of objects (Flux)

```
public Flux<UserResponse> getAllUsers() {

    return webClient

            .get()

            .uri("/api/v1/users")

            .retrieve()

            .bodyToFlux(UserResponse.class);

}



// Collect Flux into a List inside a Mono:

public Mono<List<UserResponse>> getAllUsersAsList() {

    return webClient

            .get()
```

```
            .uri("/api/v1/users")

            .retrieve()

            .bodyToFlux(UserResponse.class)

            .collectList();  // Mono<List<UserResponse>>

}
```

GET with query parameters

```
public Flux<UserResponse> searchUsers(String name, int page, int size) {

    return webClient

            .get()

            .uri(builder -> builder

                .path("/api/v1/users")

                .queryParam("name", name)

                .queryParam("page", page)

                .queryParam("size", size)

                .build())

            .retrieve()

            .bodyToFlux(UserResponse.class);

}
```

```
// With optional parameters (skip null values):

public Flux<UserResponse> filterUsers(String name, String city, Integer
minAge) {

    return webClient

            .get()

            .uri(builder -> {

                var b = builder.path("/api/v1/users");

                if (name   != null) b.queryParam("name",   name);

                if (city   != null) b.queryParam("city",   city);

                if (minAge != null) b.queryParam("minAge", minAge);

                return b.build();

            })

            .retrieve()

            .bodyToFlux(UserResponse.class);

}
```

GET with custom headers

```
public Mono<UserResponse> getAuthenticatedUser(Long id, String jwtToken) {

    return webClient
```

```
            .get()

            .uri("/api/v1/users/{id}", id)

            .header(HttpHeaders.AUTHORIZATION, "Bearer " + jwtToken)

            .header("X-Request-Id", UUID.randomUUID().toString())

            .header("X-User-Role",  "ADMIN")

            .retrieve()

            .bodyToMono(UserResponse.class);

}
```

GET ResponseEntity — access status code + headers + body

```
public Mono<ResponseEntity<UserResponse>> getUserWithMeta(Long id) {

    return webClient

            .get()

            .uri("/api/v1/users/{id}", id)

            .retrieve()

            .toEntity(UserResponse.class)   // gives you the full
ResponseEntity

            .doOnNext(response -> {

                System.out.println("Status:       " +
response.getStatusCode());
```

```
                System.out.println("Content-Type: " +
response.getHeaders().getContentType());

                System.out.println("ETag:         " +
response.getHeaders().getETag());

            });

}
```

GET with exchangeToMono — full response control

```
// Use exchangeToMono when you need the raw ClientResponse before reading the
body

public Mono<UserResponse> getUserWithCustomLogic(Long id) {

    return webClient

            .get()

            .uri("/api/v1/users/{id}", id)

            .exchangeToMono(response -> {

                if (response.statusCode().equals(HttpStatus.OK)) {

                    return response.bodyToMono(UserResponse.class);

                } else if
(response.statusCode().equals(HttpStatus.NOT_FOUND)) {

                    return Mono.error(new UserNotFoundException(id));

                } else {
```

```
                    return response.createError();  // default error handling

                }

            });

}
```

> **retrieve() vs exchangeToMono():** Use `retrieve()` for almost everything — it auto-handles 4xx/5xx with `WebClientResponseException`. Use `exchangeToMono()` only when you need to inspect the status code or headers before deciding how to read the body.

---

5. POST, PUT, PATCH, DELETE

POST — create a resource

```
public Mono<UserResponse> createUser(CreateUserRequest request) {

    return webClient

            .post()

            .uri("/api/v1/users")

            .bodyValue(request)          // serialises to JSON automatically

            .retrieve()

            .bodyToMono(UserResponse.class);

}
```

```
// With explicit Content-Type and body from a Mono:

public Mono<UserResponse> createUserFromMono(Mono<CreateUserRequest>
requestMono) {

    return webClient

            .post()

            .uri("/api/v1/users")

            .contentType(MediaType.APPLICATION_JSON)

            .body(requestMono, CreateUserRequest.class)

            .retrieve()

            .bodyToMono(UserResponse.class);

}
```

PUT — full replacement

```
public Mono<UserResponse> updateUser(Long id, UpdateUserRequest request) {

    return webClient

            .put()

            .uri("/api/v1/users/{id}", id)

            .bodyValue(request)

            .retrieve()

            .bodyToMono(UserResponse.class);
```

```
}
```

PATCH — partial update

```
public Mono<UserResponse> patchUser(Long id, PatchUserRequest request) {

    return webClient

            .patch()

            .uri("/api/v1/users/{id}", id)

            .bodyValue(request)

            .retrieve()

            .bodyToMono(UserResponse.class);

}
```

DELETE — remove a resource

```
// DELETE with no response body → Mono<Void>

public Mono<Void> deleteUser(Long id) {

    return webClient

            .delete()

            .uri("/api/v1/users/{id}", id)

            .retrieve()
```

```
            .bodyToMono(Void.class);

}



// DELETE with boolean confirmation:

public Mono<Boolean> deleteAndConfirm(Long id) {

    return webClient

            .delete()

            .uri("/api/v1/users/{id}", id)

            .retrieve()

            .toBodilessEntity()

            .map(response -> response.getStatusCode().is2xxSuccessful());

}
```

> **Mono\<Void> gotcha:** You MUST subscribe to `Mono<Void>` for the DELETE to actually execute. In a Spring WebFlux controller, just return the `Mono<Void>` — Spring subscribes for you. In a non-reactive context, call `.block()` or `.subscribe()` explicitly.

---

6. Core Operators

Operators transform, filter, and combine reactive streams. Understanding these is the key to writing clean reactive code.

map — synchronous transformation

```
// map transforms the value inside Mono or Flux synchronously

Mono<UserResponse> userMono = getUserById(1L);



// Transform UserResponse → String

Mono<String> nameMono = userMono

    .map(user -> user.getName().toUpperCase());



// Chain multiple maps

Mono<String> greeting = getUserById(1L)

    .map(user -> "Hello, " + user.getName() + "!")

    .map(String::trim)

    .map(String::toUpperCase);



// Flux map — transform every item in the stream

Flux<String> names = getAllUsers()

    .map(user -> user.getName())

    .map(String::toLowerCase);
```

flatMap — asynchronous transformation

Use `flatMap` when the transformation itself returns a `Mono` or `Flux`. Without `flatMap`, you would get `Mono<Mono<T>>` — a nested stream that never unwraps.

```
// WRONG — gives Mono<Mono<OrderResponse>> (never works as expected!)

Mono<Mono<OrderResponse>> wrong = getUserById(1L)

    .map(user -> getLatestOrder(user.getId()));   // map + async = nested!



// CORRECT — flatMap flattens the inner Mono

Mono<OrderResponse> correct = getUserById(1L)

    .flatMap(user -> getLatestOrder(user.getId()));



// Flux flatMap — each item triggers an async call, all run concurrently

Flux<OrderList> allOrders = getAllUsers()

    .flatMap(user -> getOrders(user.getId()));

// All getOrders() calls fire concurrently — fastest



// flatMapSequential — concurrent execution but preserves input ORDER

Flux<OrderList> orderedOrders = getAllUsers()

    .flatMapSequential(user -> getOrders(user.getId()));
```

```
// concatMap — sequential (one at a time), preserves order
Flux<OrderList> sequential = getAllUsers()
    .concatMap(user -> getOrders(user.getId()));
// Slowest — waits for each before starting the next
```

filter, take, skip

```
Flux<UserResponse> users = getAllUsers();


// filter — keep only active users
Flux<UserResponse> activeUsers = users
    .filter(user -> user.isActive());


// filter with multiple conditions
Flux<UserResponse> premiumUsers = users
    .filter(user -> user.isActive() && user.isPremium());


// take — first N items only
Flux<UserResponse> first10 = users.take(10);
```

```
// skip — skip first N items
Flux<UserResponse> afterFirst5 = users.skip(5);


// takeWhile — take while condition is true, stop when false
Flux<UserResponse> untilInactive = users
    .takeWhile(user -> user.isActive());
```

doOnNext, doOnError, doOnComplete — side effects

```
// doOnNext — side effect without changing the stream value
getUserById(1L)
    .doOnNext(user -> log.info("Fetched user: {}", user.getName()))
    .doOnNext(user -> metrics.incrementUserFetch())
    .map(user -> user.getName());
// The user object is unchanged — doOnNext just runs the lambda


// doOnError — log errors without handling them
getUserById(1L)
    .doOnError(ex -> log.error("Failed to fetch user: {}", ex.getMessage()))
```

```
    .onErrorReturn(UserResponse.defaultUser());



// doOnComplete — runs when the stream finishes (Flux only)

getAllUsers()

    .doOnComplete(() -> log.info("All users fetched"))

    .collectList();
```

collectList, collectMap

```
// Collect Flux → List inside a Mono

Mono<List<UserResponse>> userList = getAllUsers()

    .collectList();



// Collect Flux → Map inside a Mono (key by user ID)

Mono<Map<Long, UserResponse>> userMap = getAllUsers()

    .collectMap(UserResponse::getId);



// Collect Flux → Map with value transformation

Mono<Map<Long, String>> idToName = getAllUsers()

    .collectMap(
```

```
        UserResponse::getId,        // key extractor

        UserResponse::getName       // value extractor

    );
```

timeout — fail if too slow

```
// Fail with TimeoutException if no response within 3 seconds

getUserById(1L)

    .timeout(Duration.ofSeconds(3));




// Timeout with a fallback Mono

getUserById(1L)

    .timeout(

        Duration.ofSeconds(3),

        Mono.just(UserResponse.defaultUser())  // fallback if timeout

    );
```

block — blocking extraction (use carefully)

```
// block() waits synchronously and returns the value

// ONLY acceptable in non-reactive contexts (Spring MVC, tests)
```

```
UserResponse user = getUserById(1L)

    .block();  // blocks the current thread



// block() with timeout (throws TimeoutException if exceeded)

UserResponse user2 = getUserById(1L)

    .block(Duration.ofSeconds(5));



// blockFirst / blockLast for Flux

UserResponse first = getAllUsers().blockFirst();

List<UserResponse> all = getAllUsers().collectList().block();
```

> **NEVER call block() inside a WebFlux controller or a Netty event loop thread.** It will cause a deadlock. The `block()` call is only safe in Spring MVC servlet threads or test code.

---

7. Error Handling

onStatus — map HTTP 4xx/5xx to domain exceptions

```
public Mono<UserResponse> getUserById(Long id) {

    return webClient

            .get()
```

```
            .uri("/api/v1/users/{id}", id)

            .retrieve()

            // Handle 404 specifically

            .onStatus(HttpStatus::is4xxClientError, response ->

                response.bodyToMono(String.class)

                    .flatMap(body -> Mono.error(

                        new UserNotFoundException("User " + id + " not found:
" + body)

                    )))

            // Handle 5xx — server is down

            .onStatus(HttpStatus::is5xxServerError, response ->

                Mono.error(new ServiceUnavailableException("User service
unavailable")))

            .bodyToMono(UserResponse.class);

}
```

onErrorReturn — return a default value on error

```
// Return a default UserResponse on ANY error

getUserById(1L)

    .onErrorReturn(new UserResponse("unknown"));
```

```
// Return default only for a SPECIFIC exception type
getUserById(1L)
    .onErrorReturn(UserNotFoundException.class,
                   new UserResponse("Guest User"));


// Return default only when a CONDITION is true
getUserById(1L)
    .onErrorReturn(
        ex -> ex instanceof WebClientResponseException wcEx
              && wcEx.getStatusCode().value() == 404,
        UserResponse.defaultUser()
    );
```

onErrorResume — switch to a fallback Mono on error

```
// Try cache when primary service fails
getUserById(1L)
    .onErrorResume(UserNotFoundException.class,
                   ex -> getCachedUser(1L));  // call cache instead
```

```
// Log then re-throw
getUserById(1L)
    .onErrorResume(ex -> {
        log.error("Failed to fetch user {}: {}", 1L, ex.getMessage());
        return Mono.error(ex);  // re-propagate after logging
    });


// Full fallback chain: try primary → try cache → return default
getUserById(1L)
    .onErrorResume(ex -> getCachedUser(1L))
    .onErrorResume(ex -> Mono.just(UserResponse.defaultUser()));
```

retryWhen — automatic retry with backoff

```
import reactor.util.retry.Retry;


// Retry up to 3 times, 500ms fixed delay between attempts
getUserById(1L)
    .retryWhen(Retry.fixedDelay(3, Duration.ofMillis(500)));
```

```
// Exponential backoff — recommended for production

getUserById(1L)

    .retryWhen(Retry.backoff(3, Duration.ofMillis(200))

        .maxBackoff(Duration.ofSeconds(5))     // cap at 5 seconds

        .jitter(0.5)                           // add 50% randomness (avoids
thundering herd)

        .filter(ex -> ex instanceof IOException   // only retry transient
errors

                    || ex instanceof TimeoutException)

        .onRetryExhaustedThrow((spec, signal) ->

            new ServiceUnavailableException("User service unreachable after 3
retries"))

    );



// Full production pattern: timeout + retry + fallback

getUserById(1L)

    .timeout(Duration.ofSeconds(3))

    .retryWhen(Retry.backoff(2, Duration.ofMillis(200)))

    .onErrorReturn(UserResponse.defaultUser());
```

8. Parallel Calls

Mono.zip — run multiple calls simultaneously

`Mono.zip` fires all Mono calls at the same time and waits for ALL to complete. This is the most powerful optimisation pattern in reactive microservices.

```
Sequential (slow):   getUserById(1L) → 300ms → getOrders(1L) → 300ms →
getBudget(1L) → 200ms

Total: 800ms




Parallel (fast):     getUserById(1L)
──────────────────────────────────────────▶ 300ms

                     getOrders(1L)
──────────────────────────────────────────▶ 300ms

                     getBudget(1L)
──────────────────────────────────────────▶ 200ms

Total: ~300ms (max of all three)

// Mono.zip — fires all 3 simultaneously, combines when all done

Mono<DashboardResponse> dashboard = Mono.zip(

    getUserById(1L),       // fires now

    getOrders(1L),         // fires now

    getBudget(1L)          // fires now

).map(tuple -> DashboardResponse.builder()
```

```
    .user(tuple.getT1())      // UserResponse
    .orders(tuple.getT2())    // List<Order>
    .budget(tuple.getT3())    // BudgetResponse
    .build());


// zip with 2 Monos — clean syntax
Mono<UserWithBudget> combined = Mono.zip(
    getUserById(1L),
    getBudget(1L),
    (user, budget) -> new UserWithBudget(user, budget)
);


// zipDelayError — collects ALL errors instead of failing on the first one
Mono.zipDelayError(getUserById(1L), getBudget(1L))
    .map(t -> new Summary(t.getT1(), t.getT2()));
```

Flux.merge vs Flux.concat

```
// merge — runs all streams CONCURRENTLY, items interleave as they arrive
Flux<Event> allEvents = Flux.merge(
```

```
    getPaymentEvents(),    // stream 1 — runs in parallel

    getAuditEvents(),      // stream 2 — runs in parallel

    getSystemEvents()      // stream 3 — runs in parallel

);

// Items arrive as they come — ORDER IS NOT GUARANTEED



// concat — runs streams ONE AFTER ANOTHER, preserves ORDER

Flux<Event> ordered = Flux.concat(

    getPage(1),   // completes first...

    getPage(2),   // then this...

    getPage(3)    // then this

);

// Sequential, slower, but items arrive in order



// mergeSequential — concurrent execution, but OUTPUT is in subscription
order

Flux<Event> concurrent = Flux.mergeSequential(

    getPaymentEvents(),

    getAuditEvents(),
```

```
    getSystemEvents()

);
```

Real-world parallel dashboard example

```
@Service

@RequiredArgsConstructor

@Slf4j

public class DashboardService {



    private final WebClient webClient;



    public Mono<DashboardResponse> getDashboard(Long userId) {



        // Define all three calls — they fire simultaneously when subscribed

        Mono<UserResponse>   userMono     = fetchUser(userId);

        Mono<List<Expense>>  expensesMono = fetchExpenses(userId);

        Mono<BudgetResponse> budgetMono   = fetchBudget(userId);



        return Mono.zip(userMono, expensesMono, budgetMono)
```

```
                .map(tuple -> {

                    UserResponse   user     = tuple.getT1();

                    List<Expense>  expenses = tuple.getT2();

                    BudgetResponse budget   = tuple.getT3();



                    double totalSpent = expenses.stream()

                            .mapToDouble(Expense::amount)

                            .sum();



                    return DashboardResponse.builder()

                            .user(user)

                            .expenses(expenses)

                            .budget(budget)

                            .totalSpent(totalSpent)

                            .remainingBudget(budget.limit() - totalSpent)

                            .build();

                })

                .doOnError(ex -> log.error("Dashboard failed for userId={}:
{}",
```

```
                                        userId, ex.getMessage()))
            .onErrorResume(ex -> Mono.error(
                new DashboardException("Dashboard unavailable", ex)));
    }

    private Mono<UserResponse> fetchUser(Long id) {
        return webClient.get()
            .uri("/api/v1/users/{id}", id)
            .retrieve()
            .onStatus(HttpStatus::is4xxClientError,
                      res -> Mono.error(new UserNotFoundException(id)))
            .bodyToMono(UserResponse.class)
            .timeout(Duration.ofSeconds(3));
    }

    private Mono<List<Expense>> fetchExpenses(Long userId) {
        return webClient.get()
            .uri(b -> b.path("/api/v1/expenses")
                       .queryParam("userId", userId)
```

```
                            .build())

                .retrieve()

                .bodyToFlux(Expense.class)

                .collectList()

                .timeout(Duration.ofSeconds(5));

    }



    private Mono<BudgetResponse> fetchBudget(Long userId) {

        return webClient.get()

                .uri("/api/v1/budgets/{userId}", userId)

                .retrieve()

                .bodyToMono(BudgetResponse.class)

                .onErrorReturn(BudgetResponse.unlimited())  // budget is
optional

                .timeout(Duration.ofSeconds(3));

    }

}
```

9. WebClient in Spring Boot

Reactive controller — return Mono/Flux directly

In a Spring WebFlux controller, return `Mono` or `Flux` directly from your handler methods. Spring subscribes for you — never call `.block()` in a controller.

```
@RestController

@RequestMapping("/api/v1")

@RequiredArgsConstructor

public class DashboardController {



    private final DashboardService dashboardService;

    private final UserServiceClient userServiceClient;



    // Return Mono — Spring WebFlux subscribes automatically

    @GetMapping("/dashboard/{userId}")

    public Mono<DashboardResponse> getDashboard(@PathVariable Long userId) {

        return dashboardService.getDashboard(userId);

        // DO NOT call .block() here — Spring handles subscription

    }
```

```
// Return Flux — streams items one at a time (chunked response)

@GetMapping("/users")

public Flux<UserResponse> getAllUsers() {

    return userServiceClient.getAllUsers();

}



// Server-Sent Events — real-time streaming to browser

@GetMapping(value = "/events/{userId}", produces = "text/event-stream")

public Flux<ServerSentEvent<ExpenseEvent>> streamEvents(

        @PathVariable Long userId) {

    return dashboardService.getEventStream(userId)

            .map(event -> ServerSentEvent.<ExpenseEvent>builder()

                    .id(String.valueOf(event.getId()))

                    .event("expense-created")

                    .data(event)

                    .build());

}



// Reactive POST — request body is deserialized reactively
```

```
    @PostMapping("/users")

    public Mono<ResponseEntity<UserResponse>> createUser(

            @RequestBody Mono<CreateUserRequest> requestMono) {

        return requestMono

                .flatMap(userServiceClient::createUser)

                .map(created -> ResponseEntity

                        .created(URI.create("/api/v1/users/" +
created.getId()))

                        .body(created));

    }

}
```

WebClient in a Spring MVC (non-reactive) service

Even in a traditional Spring MVC app, WebClient is preferred over RestTemplate. Call `.block()` at the boundary between reactive and imperative code.

```
@Service

@RequiredArgsConstructor

@Slf4j

public class ExpenseServiceImpl implements ExpenseService {
```

```
    private final WebClient webClient;




    @Override

    public UserResponse getUserById(Long id) {

        // Build the full reactive pipeline...

        return webClient.get()

                .uri("/api/v1/users/{id}", id)

                .retrieve()

                .onStatus(HttpStatus::is4xxClientError,

                          res -> Mono.error(new UserNotFoundException(id)))

                .onStatus(HttpStatus::is5xxServerError,

                          res -> Mono.error(new
ServiceUnavailableException()))

                .bodyToMono(UserResponse.class)

                .timeout(Duration.ofSeconds(5))

                .retryWhen(Retry.backoff(2, Duration.ofMillis(200)))

                .doOnError(ex -> log.warn("User fetch failed: {}",
ex.getMessage()))

                .block(Duration.ofSeconds(10));

        // .block() is acceptable here — this is a Spring MVC servlet thread
```

```
    }

}
```

Global WebClient filter — add auth headers to all requests

```
@Configuration

public class WebClientConfig {



    @Value("${app.service-secret}")

    private String serviceSecret;



    @Bean

    public WebClient securedWebClient() {

        return WebClient.builder()

                .baseUrl("http://user-service")

                // Filter runs on every request

                .filter((request, next) -> {

                    ClientRequest modified = ClientRequest.from(request)

                            .header("X-Service-Auth", serviceSecret)

                            .header("X-Request-Id",
UUID.randomUUID().toString())
```

```
                            .build();

                    return next.exchange(modified);

                })

                // Logging filter

                .filter((request, next) -> {

                    log.debug("→ {} {}", request.method(), request.url());

                    return next.exchange(request)

                            .doOnNext(res ->

                                log.debug("← {} {}", res.statusCode(),
request.url()));

                })

                .build();

    }

}
```

---

10. Testing Reactive Code

StepVerifier — the reactive testing tool

```
import reactor.test.StepVerifier;
```

```
@ExtendWith(MockitoExtension.class)

class UserServiceClientTest {



    @Mock

    private WebClient webClient;



    @InjectMocks

    private UserServiceClient userServiceClient;



    @Test

    @DisplayName("Should return user when found")

    void getUserById_success() {

        UserResponse expectedUser = new UserResponse(1L, "Alice");



        // Mock the WebClient chain

        when(webClient.get()).thenReturn(requestHeadersUriSpec);

        // ... set up full mock chain
```

```
        Mono<UserResponse> result = userServiceClient.getUserById(1L);



        // StepVerifier — the right way to test Mono/Flux

        StepVerifier.create(result)

                .expectNext(expectedUser)      // expect this value

                .verifyComplete();             // expect onComplete signal

    }



    @Test

    @DisplayName("Should emit all items then complete")

    void getAllUsers_success() {

        Flux<UserResponse> users = Flux.just(

            new UserResponse(1L, "Alice"),

            new UserResponse(2L, "Bob"),

            new UserResponse(3L, "Carol")

        );



        StepVerifier.create(users)

                .expectNextCount(3)           // expect exactly 3 items
```

```
                .verifyComplete();



        // Or verify each item:

        StepVerifier.create(users)

                .expectNext(new UserResponse(1L, "Alice"))

                .expectNext(new UserResponse(2L, "Bob"))

                .expectNext(new UserResponse(3L, "Carol"))

                .verifyComplete();

    }



    @Test

    @DisplayName("Should throw UserNotFoundException for 404")

    void getUserById_notFound_throwsException() {

        // Simulate a 404 response

        Mono<UserResponse> result = Mono.error(new
UserNotFoundException(999L));



        StepVerifier.create(result)

                .expectError(UserNotFoundException.class)  // expect this
error
```

```
                .verify();

    }

}
```

WireMock — integration testing with a real HTTP server

```
@SpringBootTest

@AutoConfigureWireMock(port = 0)  // random port

class UserServiceClientIntegrationTest {



    @Autowired

    private UserServiceClient userServiceClient;



    @Test

    @DisplayName("Should deserialise JSON response correctly")

    void getUserById_returnsDeserialised() {

        // Stub a real HTTP response

        stubFor(get(urlEqualTo("/api/v1/users/1"))

                .willReturn(aResponse()

                        .withStatus(200)
```

```
                        .withHeader("Content-Type", "application/json")

                        .withBody("""

                                {

                                  "id": 1,

                                  "name": "Alice",

                                  "email": "alice@example.com"

                                }

                                """)));



        // Call the real WebClient against WireMock

        StepVerifier.create(userServiceClient.getUserById(1L))

                .assertNext(user -> {

                    assertThat(user.getId()).isEqualTo(1L);

                    assertThat(user.getName()).isEqualTo("Alice");

                })

                .verifyComplete();

    }



    @Test
```

```
    @DisplayName("Should throw UserNotFoundException on 404")

    void getUserById_404_throwsException() {

        stubFor(get(urlEqualTo("/api/v1/users/999"))

                .willReturn(aResponse().withStatus(404)));



        StepVerifier.create(userServiceClient.getUserById(999L))

                .expectError(UserNotFoundException.class)

                .verify();

    }

}
```

---

11. Best Practices Cheat Sheet

Do this

```
// ✅ Return Mono/Flux from WebFlux controllers

@GetMapping("/{id}")

public Mono<UserResponse> getUser(@PathVariable Long id) {

    return userService.getUser(id);  // Spring subscribes for you

}
```

```
// ✅ Use Mono.zip() for parallel calls
Mono.zip(getUserById(id), getBudget(id), getOrders(id))
    .map(t -> buildDashboard(t.getT1(), t.getT2(), t.getT3()));


// ✅ Always set a timeout
getUserById(1L).timeout(Duration.ofSeconds(5));


// ✅ Use onStatus() for clear error mapping
.onStatus(HttpStatus::is4xxClientError,
          res -> Mono.error(new UserNotFoundException(id)))


// ✅ Use exponential backoff for retries
.retryWhen(Retry.backoff(3, Duration.ofMillis(200)).jitter(0.5))


// ✅ Use doOnNext/doOnError for logging (never for logic)
.doOnNext(user -> log.info("Fetched user: {}", user.getName()))
.doOnError(ex  -> log.error("Failed: {}", ex.getMessage()))
```

```
// ✅ Use StepVerifier for testing reactive code

StepVerifier.create(myMono)

    .expectNext(expectedValue)

    .verifyComplete();
```

Never do this

```
// ❌ Never call block() in a WebFlux controller

@GetMapping("/{id}")

public UserResponse getUser(@PathVariable Long id) {

    return userService.getUser(id).block();  // DEADLOCK risk!

}



// ❌ Never call block() on a Netty event loop thread

// (inside a reactive pipeline)

getUserById(1L).map(user -> {

    String name = getSomeOtherUser(user.getId()).block();  // DEADLOCK!

    return name;

});
```

```
// ❌ Never use map when the transformation returns a Mono

getUserById(1L).map(user -> getOrders(user.getId()));

// Returns Mono<Mono<...>> — nothing ever subscribes to inner Mono!

// Use flatMap instead.



// ❌ Never ignore error handling in production

getUserById(1L)

    .subscribe(user -> process(user));

// What happens on error? Silent failure!

// Always handle errors:

getUserById(1L)

    .subscribe(

        user  -> process(user),

        error -> log.error("Failed: ", error)

    );
```

Quick operator reference

Operator	Use case	Returns
`map(f)`	Sync transform one value	Same type wrapper
`flatMap(f)`	Async transform, f returns Mono/Flux	Flattened result

Operator	Use case	Returns
`flatMapSequential(f)`	Like flatMap but preserves order	Flattened, ordered
`filter(pred)`	Keep items matching predicate	Same type
`take(n)`	First N items only	Flux
`skip(n)`	Skip first N items	Flux
`collectList()`	Flux → Mono\<List>	`Mono<List<T>>`
`collectMap(k)`	Flux → Mono\<Map>	`Mono<Map<K,V>>`
`doOnNext(f)`	Side effect per item	Unchanged stream
`doOnError(f)`	Side effect on error	Unchanged stream
`timeout(d)`	Fail if no item in duration d	Same type
`retry(n)`	Retry n times on error	Same type
`retryWhen(spec)`	Retry with backoff strategy	Same type
`onErrorReturn(v)`	Return default on error	Same type
`onErrorResume(f)`	Switch to fallback Mono/Flux on error	Same type
`onStatus(pred, f)`	Map HTTP status to error	Same type
`block()`	Synchronously extract value	`T` (unwrapped)
`blockFirst()`	Synchronously extract first item	`T` (unwrapped)
`Mono.zip(m1, m2, ...)`	Parallel execution, combine results	`Mono<TupleN>`
`Flux.merge(f1, f2)`	Merge streams, concurrent	`Flux<T>`
`Flux.concat(f1, f2)`	Concatenate streams, sequential	`Flux<T>`

📦 Dependencies Summary

```
<!-- Spring WebFlux (includes WebClient + Reactor) -->

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-webflux</artifactId>

</dependency>



<!-- Reactor Test (for StepVerifier) -->

<dependency>

    <groupId>io.projectreactor</groupId>

    <artifactId>reactor-test</artifactId>

    <scope>test</scope>

</dependency>



<!-- WireMock (for integration tests) -->

<dependency>

    <groupId>org.springframework.cloud</groupId>

    <artifactId>spring-cloud-starter-contract-stub-runner</artifactId>

    <scope>test</scope>
```

```
</dependency>
```

WebClient + Reactor is the foundation of modern Spring Boot microservices. Master `Mono.zip()` for parallel calls, `onStatus()` for error mapping, and `retryWhen(Retry.backoff())` for resilience — these three patterns cover 90% of real-world use cases. ⚡